# 実装計画 - 法人向けAIアプリ ハンズ

作成日: 2026年2月23日 | ステータス: 計画フェーズ | 対象: ハンズ（hands）

---

## 概要

ハンズ（hands）は、法人向けAIアシスタントプラットフォームです。Slack / Teams / LINE WORKS / Web チャットを通じて、ナレッジ検索・タスク管理・ドキュメント作成・データ分析・外部システム連携を自律的に実行します。

---

## 実装フェーズ

### Phase 1: コアエンジン（Mastra ベース）

- Mastra フレームワークのセットアップ（Hono ベース）
- Agent 定義（メインエージェント、ツール呼び出し、マルチステップ推論）
- Tool 定義（ナレッジ検索、タスク管理、ドキュメント作成、データ分析）
- Workflow 定義（定期実行、承認フロー、長時間処理）
- Memory 設定（会話履歴、ユーザーコンテキスト）

### Phase 2: チャネル連携

- Slack Bolt SDK 統合（Socket Mode / HTTP）
- Teams Bot Framework 統合
- LINE WORKS Bot API 統合
- Web チャット UI（Hono + SSE ストリーミング）
- チャネル横断のセッション管理

### Phase 3: ナレッジ管理

- ドキュメント取り込み（PDF / DOCX / TXT / MD）
- チャンク分割 + 埋め込み生成（OpenAI text-embedding-3-small）
- ベクトル検索（pgvector）
- ハイブリッド検索（ベクトル + キーワード）
- ナレッジベース管理 UI

### Phase 4: 外部システム連携

- Google Workspace（Calendar / Drive / Gmail）
- Microsoft 365（Outlook / OneDrive / SharePoint）
- kintone
- Salesforce
- 汎用 Webhook / REST API

### Phase 5: 管理・運用

- 管理ダッシュボード（Next.js）
- 利用ログ・監査ログ
- 使用量・コスト管理
- 組織・ユーザー管理
- 課金（Stripe）

### Phase 6: 本番化

- Docker Compose 構成
- Cloud Run デプロイ
- CI/CD（GitHub Actions）
- 監視・アラート
- バックアップ・DR

---

## アーキテクチャ概要

---

## 技術スタック

| カテゴリ | 採用技術 |
|---|---|
| ランタイム | Node.js 22 |
| フレームワーク | Hono |
| AI フレームワーク | Mastra |
| LLM | OpenAI GPT-4o / Anthropic Claude |
| 埋め込み | OpenAI text-embedding-3-small |
| データベース | PostgreSQL + pgvector |
| キャッシュ | Redis |
| ファイルストレージ | Cloud Storage |
| Slack | Slack Bolt SDK |
| Teams | Bot Framework |
| LINE WORKS | LINE WORKS Bot API |
| Web UI | Next.js + Tailwind CSS |
| 課金 | Stripe |
| デプロイ | Docker / Cloud Run |
| CI/CD | GitHub Actions |
| 監視 | Cloud Logging / Cloud Monitoring |

---

## ディレクトリ構成

```
hands/
├── apps/
│   ├── server/          # Hono + Mastra サーバー
│   ├── admin/           # 管理ダッシュボード (Next.js)
│   ├── slack/           # Slack Bolt ハンドラー
│   ├── teams/           # Teams Bot ハンドラー
│   ├── lineworks/       # LINE WORKS Bot ハンドラー
│   └── web-chat/        # Web チャット UI
├── packages/
│   ├── shared/          # 共通型・ユーティリティ
│   ├── db/              # DB スキーマ・マイグレーション
│   ├── knowledge/       # ナレッジ管理ライブラリ
│   ├── integrations/    # 外部システム連携
│   ├── billing/         # 課金ロジック
│   ├── usage/           # 使用量トラッキング
│   ├── notifications/   # 通知
│   ├── ui/              # 共通 UI コンポーネント
│   ├── eslint-config/   # ESLint 設定
│   ├── prettier-config/ # Prettier 設定
│   ├── tsconfig/        # TSconfig 設定
│   ├── vitest-config/   # Vitest 設定
│   ├── test-helpers/    # テストヘルパー
│   ├── fixtures/        # テストフィクスチャ
│   ├── mocks/           # モック
│   ├── e2e/             # E2E テスト
│   ├── load-test/       # 負荷テスト
│   ├── security-test/   # セキュリティテスト
│   ├── seed/            # シードデータ
│   ├── docker/          # Docker 設定
│   ├── deploy/          # デプロイ設定
│   ├── ci/              # CI 設定
│   ├── monitoring/      # 監視設定
│   ├── backup/          # バックアップ
│   ├── docs/            # ドキュメント
│   ├── scripts/         # スクリプト
│   ├── tasks/           # タスク管理
│   ├── .claude/         # Claude 設定
│   ├── .agent/          # エージェント設定
│   ├── .vscode/         # VSCode 設定
│   ├── .devcontainer/   # Dev Container 設定
│   ├── .github/         # GitHub 設定
│   ├── .husky/          # Husky フック
│   ├── .mcp.log         # MCP ログ
│   ├── .mcp.log.1       # MCP ログ (ローテーション)
│   ├── .mcp.log.2       # MCP ログ (ローテーション)
│   ├── .mcp.log.3       # MCP ログ (ローテーション)
│   ├── .mcp.log.4       # MCP ログ (ローテーション)
│   ├── .mcp.log.5       # MCP ログ (ローテーション)
│   ├──
```